
ISyE 6740 – Spring 2026

Final Project Report

Team Member Names: Pramodh Aryasomayajula, Keshav Shah

Project Title: Predicting Environmental Compliance Violations at Small Manufacturing Facilities Using EPA ECHO Data

1 Problem Statement

The U.S. Environmental Protection Agency (EPA) oversees roughly 800,000 facilities under laws like the Clean Air Act (CAA), Clean Water Act (CWA), and Resource Conservation and Recovery Act (RCRA). But regulatory agencies only have so many inspectors and so much budget, and smaller facilities tend to get less attention than those classified as major sources. Small manufacturers, meaning those in NAICS sectors 31–33 that do not have a major regulatory designation, make up a large chunk of the regulated population. Compliance costs for these firms are significant too, exceeding \$50,000 per employee per year according to some estimates [1].

The question we are trying to answer is: can we predict which small manufacturing facilities will have environmental compliance violations in the next year, based on their past compliance patterns and facility characteristics? If we can build a model that does this reliably, it could help regulators decide where to focus their inspections, and also help manufacturers understand what factors are associated with non-compliance.

We treat this as a binary classification problem. Using the EPA’s Enforcement and Compliance History Online (ECHO) database, we construct a pseudo-temporal prediction setup by splitting each facility’s 12-quarter compliance history string into a 2-year feature window and a 1-year target window. We then compare four classifiers (Logistic Regression, Random Forest, XGBoost, and SVM) and evaluate them with AUC-ROC, AUC-PR, and bootstrap confidence intervals. As far as we know, this kind of supervised learning approach has not really been applied to the small manufacturer segment of ECHO data for violation prediction.

2 Data Description and Cleaning

2.1 Data Source

The data for this project comes from the EPA ECHO Exporter, which is a weekly-updated bulk download containing facility-level summaries for all regulated establishments in the U.S. The full export file has 3,091,085 facilities across 133 columns, covering things like facility identification, geographic location, what regulatory programs they participate in, inspection and enforcement history, penalties, and quarterly compliance status.

2.2 Population Filtering

We applied the following filters to get to our target population:

1. **Industry:** Kept facilities with at least one NAICS code in the manufacturing sectors (31, 32, or 33).
2. **Size proxy:** Removed facilities flagged as major (`FAC_MAJOR_FLAG = Y`). We use non-major status as a rough proxy for smaller establishments since exact employee counts are not available in ECHO.
3. **Activity:** Only kept active facilities (`FAC_ACTIVE_FLAG = Y`).
4. **History completeness:** Required the full 12-quarter compliance history string to be present.

After all of these filters, we ended up with about 120,000 small manufacturing facilities.

2.3 Target Variable Construction

The ECHO Exporter includes a field called `FAC_3YR_COMPLIANCE_HISTORY`, which is a 12-character string where each character represents one quarter’s compliance status over the past three years. Each character encodes one quarter: underscore (`_`) for no violation, `V` for violation, `S` for significant non-compliance, and `U` for unknown. Position 0 is the oldest quarter and position 11 is the most recent.

We split this string to create a pseudo-temporal prediction problem:

- **Feature window** (positions 0–7): the older 2 years, used to build our predictive features.
- **Target window** (positions 8–11): the most recent year. The binary target is 1 if any character in this window is `V` or `S`, and 0 otherwise.

This lets us simulate the real-world task of predicting next-year violations from historical data, even though we’re working with a single cross-sectional snapshot. The resulting positive class rate is about 8.1%, which is the moderate class imbalance we expected from our proposal.

2.4 Feature Engineering

We built features across four main categories:

Table 1: Feature categories and descriptions

Category	Features	Description
Compliance History	<code>n_nc_qtrs</code> , <code>n_snc_qtrs</code> , <code>violation_trend</code> , <code>last_nc_distance</code> , <code>longest_clean_streak</code> , <code>recency_weighted</code>	Parsed from the 8-quarter feature window of the general, CAA, CWA, and RCRA history strings
Industry	<code>naics_2digit</code> , <code>naics_3digit</code>	Extracted from the primary manufacturing NAICS code
Geography & Demographics	<code>fac_state</code> , <code>fac_epa_region</code> , <code>fac_lat</code> , <code>fac_long</code> , <code>percent_minority</code> , <code>pop_den</code>	State, EPA region, coordinates, and census demographics
Regulatory Profile	Program flags (CAA, CWA, RCRA, SDWA, TRI, GHG), <code>n_programs</code> , inspection counts, penalty counts	How many and which regulatory programs apply, plus historical inspection and enforcement activity

2.5 Data Leakage Prevention

A number of fields in ECHO directly encode the full 12-quarter compliance history or current compliance status, which would leak the target variable if we included them. We excluded the following fields:

- `FAC_QTRS_WITH_NC` (exactly equals the count of V/S across all 12 quarters)
- `FAC_COMPLIANCE_STATUS`, `FAC_SNC_FLG`
- All program-specific `*_COMPLIANCE_STATUS` and `*_SNC_FLAG` fields
- `FAC_DAYS_LAST_INSPECTION`

We did keep cumulative lifetime counts like `FAC_INSPECTION_COUNT`, but ran a sensitivity analysis to check whether those introduce any soft leakage since they might include activity from the target period.

3 Methodology

3.1 Preprocessing

We used median imputation for the small number of missing values in `FAC_PERCENT_MINORITY` and `FAC_POP_DEN`, and filled program-specific counts with zero for facilities not in that program. For high-cardinality categoricals (`naics_3digit` with around 80 categories, `fac_state` with around 50), we used scikit-learn’s `TargetEncoder`, which handles cross-fitting internally to prevent leakage during cross-validation. Lower-cardinality categoricals (`naics_2digit`, `fac_epa_region`) were one-hot encoded. All numeric features were standardized with `StandardScaler`. The whole thing was wrapped in a `ColumnTransformer` so that transformations only get fit on training data.

We split the data 80/20 into training and test sets, stratified by the target variable.

3.2 Models

We trained four classifiers, picking models that cover a range of complexity and interpretability:

Logistic Regression (L1). This serves as our interpretable baseline. The L1 penalty does feature selection by pushing less useful coefficients to zero. We tuned the regularization strength $C \in \{0.001, 0.01, 0.1, 1, 10\}$ using 5-fold stratified CV with AUC-ROC scoring, and set class weights to `balanced` to handle the imbalance.

Random Forest. An ensemble of decision trees that works well with mixed feature types and gives us built-in feature importance. We tuned the number of trees $\{200, 500\}$, max depth $\{5, 10, 20\}$, and minimum samples per leaf $\{5, 20, 50\}$ via grid search with balanced class weights.

XGBoost. Gradient-boosted trees, which tend to do very well on tabular data [2]. We used `scale_pos_weight` for the class imbalance and tuned depth, learning rate, number of estimators, subsample ratio, and column sample ratio through randomized search over 30 configurations.

SVM (RBF Kernel). We included this to see whether a kernel-based decision boundary could improve classification. The main issue is that the RBF kernel has $O(n^2)$ memory requirements for the kernel matrix, so we could not train on the full dataset. We instead trained on a stratified subsample of 12,000 rows and evaluated on the full test set. We tuned $C \in \{0.1, 1, 10\}$ and

$\gamma \in \{\text{scale}, 0.01, 0.001\}$.

3.3 Class Imbalance Handling

With about 8% of facilities in the positive class, we compared three strategies using the best XGBoost config: (1) no weighting at all, (2) class weighting through `scale_pos_weight`, and (3) SMOTE oversampling applied inside the cross-validation loop using `imblearn.pipeline.Pipeline` so that synthetic samples only come from training folds.

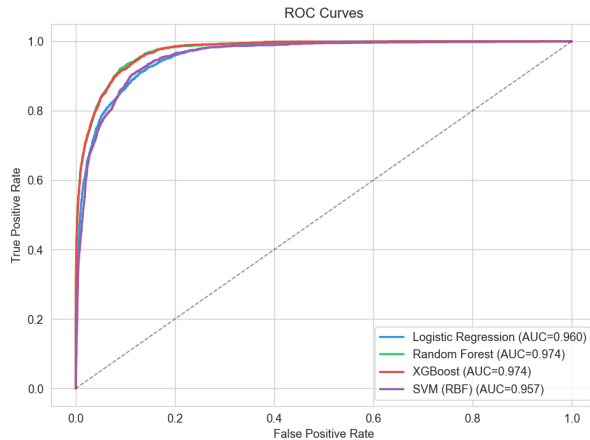
4 Results and Evaluation

4.1 Model Comparison

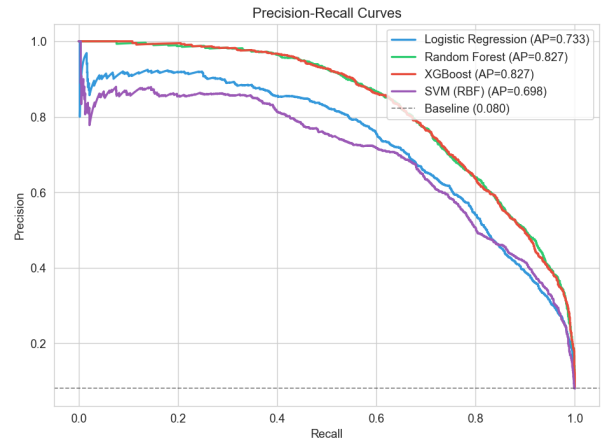
All models were evaluated on the held-out test set. We report AUC-ROC with 95% bootstrap confidence intervals (1,000 resamples), AUC-PR (average precision), and precision/recall/F1 at the threshold that maximizes Youden’s J statistic.

Table 2: Test set performance. Confidence intervals are from 95% bootstrap.

Model	AUC-ROC [95% CI]	AUC-PR	Precision	Recall	F1
Logistic Regression	0.960 [0.956, 0.963]	0.733	0.380	0.910	0.536
Random Forest	0.974 [0.972, 0.977]	0.827	0.471	0.922	0.623
XGBoost	0.974 [0.971, 0.977]	0.827	0.463	0.917	0.616
SVM (RBF)	0.957 [0.953, 0.961]	0.698	0.414	0.903	0.567



(a) ROC curves



(b) Precision-Recall curves

Figure 1: ROC and PR curves for all four models. Dashed line in (a) is random; in (b) it shows the baseline positive rate.

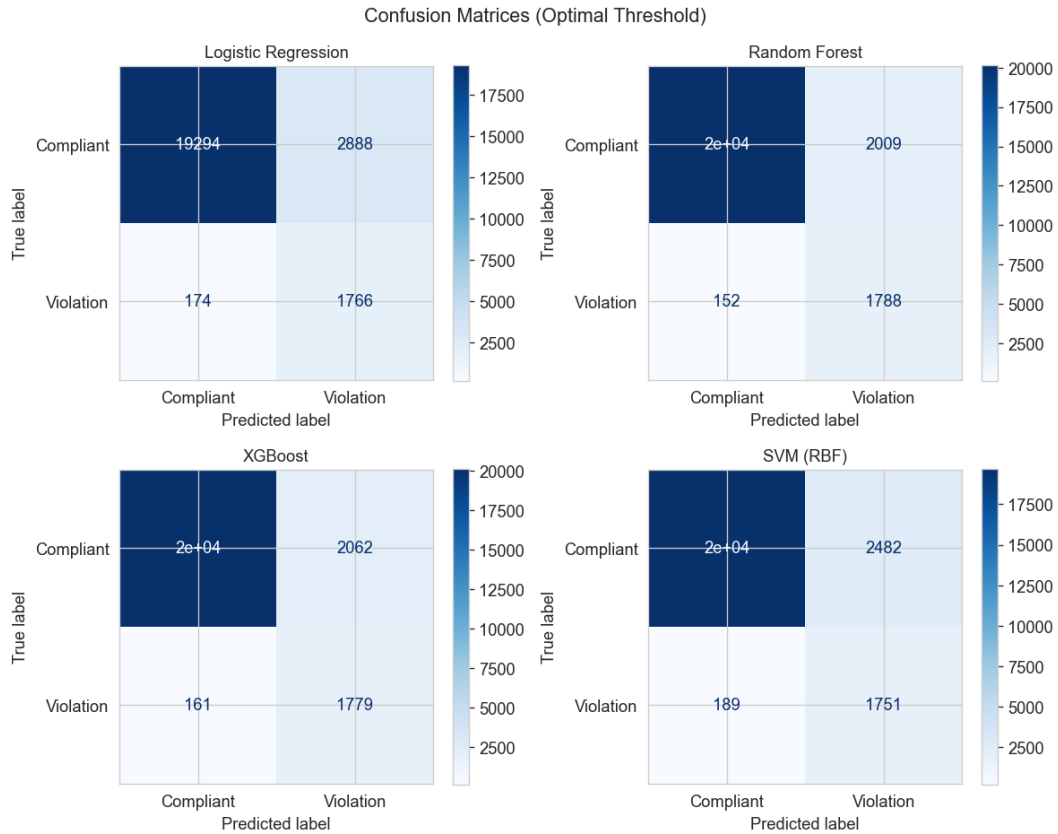


Figure 2: Confusion matrices at the Youden's J optimal threshold for each model.

4.2 Feature Importance

We looked at feature importance across three of the models to see which facility traits matter most for predicting violations:

Feature Importance Comparison (Top 15 Features)

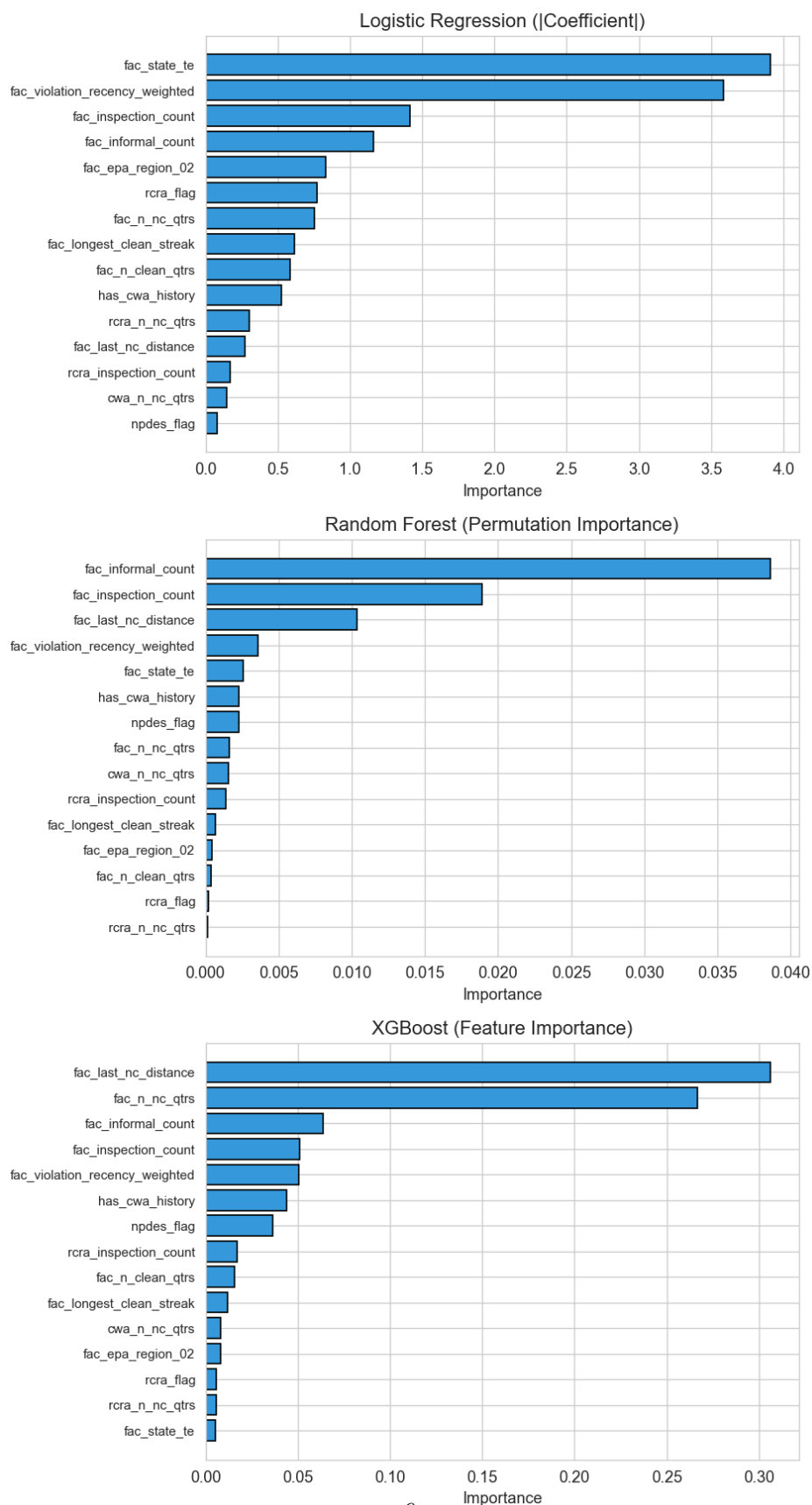


Figure 3: Feature importance across models. LR uses absolute coefficient values, RF uses permutation importance, and XGBoost uses gain-based importance.

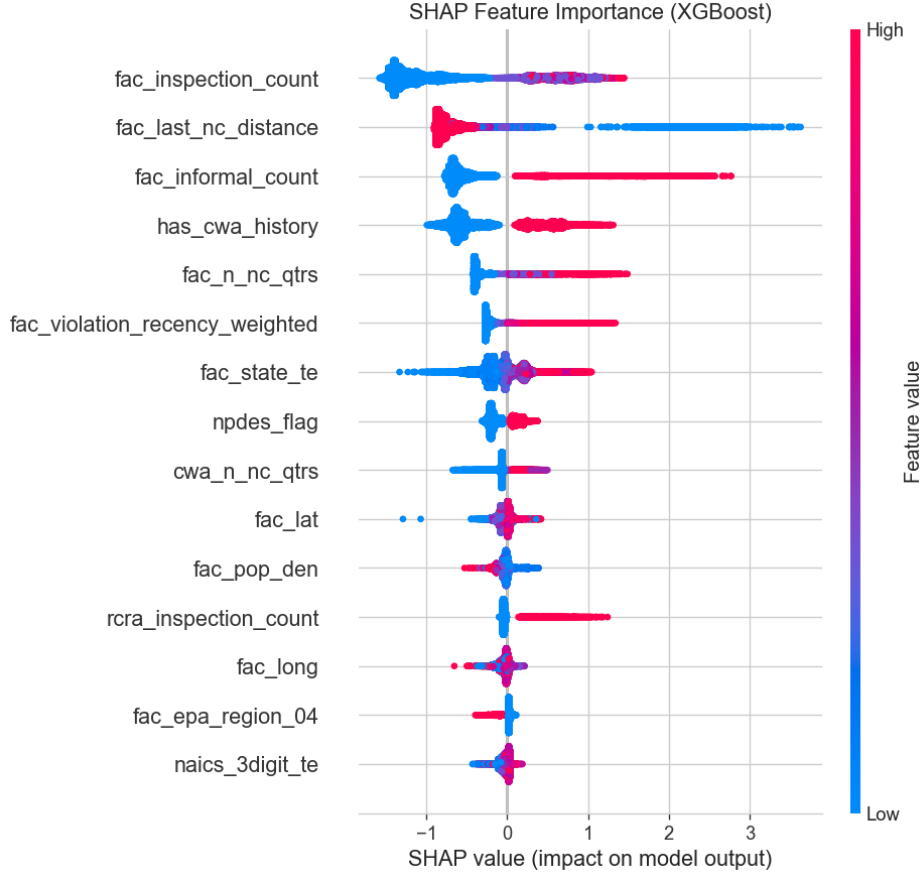


Figure 4: SHAP beeswarm plot for XGBoost, showing the direction and magnitude of each feature’s effect on predictions.

Across all three models, the strongest predictors are the compliance history features: how many non-compliant quarters appear in the feature window, the recency-weighted violation score, and how long ago the last non-compliance event was. This makes sense and lines up with the environmental compliance literature, which generally finds that past violations are the best predictor of future ones. The number of active regulatory programs also ranks high, probably because facilities under more programs just have more opportunities for a violation to get detected.

4.3 Class Imbalance Experiments

Table 3: Comparing class imbalance strategies (XGBoost)

Strategy	AUC-ROC	AUC-PR
No Weighting	0.974	0.833
Class Weighting	0.974	0.827
SMOTE	0.972	0.814

4.4 Sensitivity Analysis

To check for potential soft leakage from the cumulative lifetime count features (which might include activity during the target period), we retrained XGBoost with and without those features. Dropping the cumulative counts lowered AUC-ROC from 0.974 to 0.926, a drop of 0.048. This is not nothing, but the model without those counts still gets strong discrimination ($\text{AUC} > 0.92$), which tells us that the compliance history string features, which have a clean temporal split, are carrying most of the predictive signal. The cumulative counts add useful information on top of that rather than being pure leakage.

4.5 Error Analysis

We looked at the false negatives (facilities that violated but got predicted as compliant) and false positives (predicted to violate but stayed compliant) from the best model. The false negatives tend to be facilities that had fewer prior violations and less regulatory activity in their feature window. Basically they look a lot like compliant facilities because there just isn't much signal in their history. This means the model's main weakness is predicting first-time or sporadic violators, which is inherently hard when the approach relies on historical patterns.

5 Discussion and Conclusion

5.1 Key Findings

Our results show that compliance violations at small manufacturing facilities can be predicted with strong discrimination (AUC-ROC at or above 0.957 across all models) using historical compliance patterns and facility characteristics. Random Forest got the highest test AUC-ROC of 0.974 [0.972, 0.977], with XGBoost performing essentially the same at 0.974 [0.971, 0.977]. Both did noticeably better than Logistic Regression (0.960) and SVM (0.957), which is consistent with what the literature says about ensemble methods on tabular data [2].

The SHAP analysis shows that the most important predictors are inspection count, distance since the last non-compliance event, informal enforcement action count, CWA program participation, and the count of non-compliant quarters in the feature window. The directions make intuitive sense: more inspections and more recent violations push the predicted violation probability up. The number of active regulatory programs also has a clear monotonic relationship with violation rates, going from about 4% for facilities in just 1 program up to 43% for those in 5 programs.

For the class imbalance experiments, all three strategies (no weighting, class weighting, SMOTE) gave similar AUC-ROC around 0.974. This suggests the models handle the 8% imbalance fine on their own. No weighting actually came out slightly ahead on AUC-PR (0.833 vs 0.827 and 0.814).

5.2 Limitations

There are some limitations worth noting:

1. **Cross-sectional snapshot:** Our pseudo-temporal split operates within a single data export.

A proper temporal validation would use multiple quarterly snapshots, but those weren't readily available for this project.

2. **Cumulative count features:** The lifetime inspection and penalty counts might include activity from the target period. The sensitivity analysis shows the effect is there but not dominant.
3. **SVM scalability:** We had to train the RBF kernel SVM on just 12,000 rows, which might understate how well it could do with more data.
4. **Small facility proxy:** ECHO doesn't have employee counts, so we used non-major regulatory status as a proxy for facility size. This is imperfect but it's what the data gives us.

5.3 Future Work

There are a few directions we think this could go: (1) downloading multiple quarterly snapshots from ECHO to do real temporal validation, (2) pulling in the facility-level detail files from ECHO for richer inspection and enforcement features, (3) bringing in external data like Census County Business Patterns for better size estimates, and (4) potentially deploying this as a risk-scoring tool for regulatory targeting.

5.4 Division of Responsibilities

Pramodh (Data & Insights)	Keshav (Modeling & Evaluation)
Data acquisition, filtering, cleaning, and feature engineering	Preprocessing pipeline: imputation, encoding, standardization, and train/test split
Exploratory data analysis and missingness reporting	Model implementation and tuning (LR, RF, XGBoost, SVM)
Feature importance analysis and SHAP interpretation	Class imbalance experiments and evaluation metrics
Report: Introduction, Discussion, Conclusion	Report: Data, Methodology, Results
Joint: Error analysis, sensitivity analysis, report review, and final editing	

References

- [1] National Association of Manufacturers. "The Cost of Federal Regulation to the U.S. Economy, Manufacturing, and Small Business." NAM, 2023.
- [2] Chen, T. & Guestrin, C. "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [3] Davis, J. & Goadrich, M. "The Relationship Between Precision-Recall and ROC Curves." *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006.
- [4] U.S. Environmental Protection Agency. "Enforcement and Compliance History Online (ECHO)." echo.epa.gov. Accessed March 2026.

- [5] U.S. Census Bureau. “County Business Patterns: 2022.”
census.gov/programs-surveys/cbp.html.